Lab_03 Shell Scripting:

Create and run a simple shell script:

```
mkdir scripts

cd scripts

touch hello.sh
```

inside file:

```
#! /bin/bash



echo 'Hello, World!'
```

Run the script:

```
Bash hello.sh

Or

./hello.sh
```

Exercise 1:

Write a shell script that prints all your user name, current working directory, home directory and default shell. Print all the answers in a separate line.

your output should be like this:

```
Hello

My name is XYZ

My current location is XYZ

My home directory is XYZ

My default shell is XYZ
```

Using variables in bash shell scripts

Let's improve this script by using shell variables so that it greets users with their names. Edit your hello.sh script and use read command to get input from user.

```
#! /bin/bash

echo "What's your name, stranger?"

read name

echo "Hello, $name"
```

Now if you run your hello.sh script; it will prompt you for your name and then it will greet you with whatever name you provide to it

Step by step explanation of the above shell script

included the shebang line to explicitly state that we are going to use bash shell to run this script.

```
#!/bin/bash
```

Next, ask the user to enter his/her name

```
echo "What's your name, stranger?"
```

That’s just a simple echo command to print the line to the terminal.

```
read name
```

Here, we used the read command to transfer the control from running script to the user, so that the user can enter a name and then store whatever user entered, in the 'name' variable.

```
echo “Hello, $name”
```

Notice here, you have to precede the variable name with a dollar sign to get the value stored in the variable name. If you omit the dollar sign, “Hello, name” would be displayed instead.

Variables can hold different types of data; variables can store integers, strings, and characters.

```
letter=’c’

color=’blue’

year=2020
```

You can also create a constant variable, that is to say, a variable whose value will never change! This can be done by preceding your variable name with the readonly command

```
readonly PI=3.14159
```

Command substitutions

The ability to store the output of a command into a variable is called command substitution.

```
TODAY=$(date)
```

Exercise 2:

Write a shell script in that prints all the outputs for your name, section and reg_no. Print all the answers in a separate line.

your output should be like this:

```
What is your name?

XYZ ABC

Hello XYZ ABC

What is your reg_no?

44

Your reg_no is 44.
```

```
Today's date is Thursday 20 January 2022 12:55:23 PM IST
```

Passing Arguments to Bash Scripts

In the third part of the Bash Beginner Series, you'll learn to pass arguments to a bash shell script. You'll also learn about special bash shell variables.

The following script count_lines.sh will output the total number of lines that exist in whatever file the user enters:

```
#!/bin/bash


echo -n "Please enter a filename: "

read filename

nlines=$(wc -l < $filename)


echo "There are $nlines lines in $filename"
```

This script works fine; however, there is a much better alternative!

Instead of prompting the user for the filename, we can make the user simply pass the filename as a command line argument while running the script as follows:

```
./count_lines.sh file1.txt
```

The first bash argument (also known as a positional parameter) can be accessed within your bash script using the **$1** variable.

So in the count_lines.sh script, you can replace the filename variable with **$1** as follows:

```
#!/bin/bash


nlines=$(wc -l < $1)

echo "There are $nlines lines in $1"
```

Passing multiple arguments to a bash shell script

You can pass more than one argument to your bash script. In general, here is the syntax of passing multiple arguments to any bash script:

```
script.sh arg1 arg2 arg3
```

The second argument will be referenced by the **$2** variable, the third argument is referenced by **$3**, .. etc.

The $0 variable contains the name of your bash script.

Now we can edit our count_lines.sh bash script so that it can count the lines of more than one file

```
#!/bin/bash


n1=$(wc -l < $1)

n2=$(wc -l < $2)

n3=$(wc -l < $3)


echo "There are $n1 lines in $1"

echo "There are $n2 lines in $2"

echo "There are $n3 lines in $3"
```

Special variables in Bash shell

Special Variable	Description
$0	The name of the bash script.
$1, $2...$n	The bash script arguments.
$$	The process id of the current shell.
$#	The total number of arguments passed to the script.

$@	The value of all the arguments passed to the script.
$?	The exit status of the last executed command.
$!	The process id of the last executed command.

To see these special variables in action; take a look at the following variables.sh bash script:

```
#!/bin/bash


echo "Name of the script: $0"

echo "Total number of arguments: $#"

echo "Values of all the arguments: $@"
```

Exercise 3:

Write a shell script that takes 3 arguments and prints them in the reverse order.

Since if-else is not covered at this stage, no need to make a check on number of arguments. Only make sure to provide 3 arguments to the script.

Sample output:

```
./reverse.sh Ritchie Stallman Torvalds

Output:

Original Input - Ritchie Stallman Torvald

Reversed Output - Torvalds Stallman Ritchie
```

Exercise 4:

Write a shell script that accepts a directory name called hello and creates it if it doesn't exist already. Switch to this directory and create a new file named new.txt.

Use the -p option of the mkdir command

Using Arrays in Bash

Let's say you want to create a bash script *timestamp.sh* that updates the timestamp of five different files.

you can create an array named files that stores all the five filenames

```
files=("f1.txt" "f2.txt" "f3.txt" "f4.txt" "f5.txt")
```

Accessing array elements in bash

The first element of an array starts at index **0** and so to access the nth element of the array you use the **n -1** index.

```
echo ${files[1]}
```

The following bash script *reverse.sh* would print out all the five values in your files array in reversed order, starting with the last array element:

```
#!/bin/bash


files=("f1.txt" "f2.txt" "f3.txt" "f4.txt" "f5.txt")


echo ${files[4]}
echo ${files[3]}
echo ${files[2]}
echo ${files[1]}
echo ${files[0]}
```

Try the following:

```
echo ${files[*]}
echo ${#files[@]}
files [0]="a.txt"
```

Adding array elements in bash

```
distros=("Ubuntu" "Red Hat" "Fedora")
echo ${ distros [*]}
distros+=("Kali")
echo ${ distros [*]}
```

Deleting array elements in bash

```
num=(1 2 3 4 5)
echo ${num[*]}
unset num[2]
echo ${num[*]}
```

You can also delete the whole num array in the same way:

```
unset num
```

Creating hybrid arrays with different data types

```
#!/bin/bash


user=("john" 122 "sudo,developers" "bash")


echo "User Name: ${user[0]}"
echo "User ID: ${user[1]}"
echo "User Groups: ${user[2]}"
echo "User Shell: ${user[3]}"
```

Exercise 5:

Write a shell script in that uses an array to store the names of the seven colors of the rainbow and prints it. (Violet, Indigo, Blue, Green, Yellow, Orange and Red).

Swap any two color in the array.

Remove 3rd color from the array.

Add new color to the array.

Using Arithmetic Operators in Bash Scripting

performing arithmetic operations like addition, subtraction, multiplication and division in bash scripts.

Operator	**Description**
+	Addition
-	Substraction
*	Multiplication
/	Integer division (without decimal numbers)
%	Modulus division (gives only remainder)
**	exponentiation (x to the power y)

Addition;

```
#!/bin/bash
```

```
Echo “Enter value for a: “
Read a
Echo “Enter value for b: “
Read b
total=$(($a + $b))


echo "Total size is: $total"
```

Exercise 6:

Write a shell script that accepts the sides of the rectangle and prints its area and perimeter.

Area of a rectangle is the product of length and width. Perimeter is twice the sum of length and width.

Output: Check with length 10 and width 5. Area should be 50 and perimeter is 30.

Decision Making With If Else and Case Statements

The most fundamental construct in any decision-making structure is an if condition. The general syntax of a basic if statement is as follows:

```
if [ condition ]; then
  your code
fi
```

The if statement is closed with a fi (reverse of if).

Pay attention to white space!

- There must be a space between the opening and closing brackets and the condition you write. Otherwise, the shell will complain of error.
- There must be space before and after the conditional operator (=, ==, <= etc). Otherwise, you'll see an error like "unary operator expected".

Now, let's create an example script root.sh. This script will echo the statement “you are root” only if you run the script as the root user:

```
#!/bin/bash


if [ $(whoami) = 'root' ]; then
        echo "You are root"
fi
```

Using if-else statement in bash

```
#!/bin/bash


if [ $(whoami) = 'root' ]; then
        echo "You are root"
else
        echo "You are not root"
fi
```

Using else if statement in bash

```
#!/bin/bash


AGE=$1


if [ $AGE -lt 13 ]; then
        echo "You are a kid."
elif [ $AGE -lt 20 ]; then
        echo "You are a teenager."
```

```
elif [ $AGE -lt 65 ]; then
        echo "You are an adult."
else
        echo "You are an elder."
fi
```

Using nested if statements in bash

```
#!/bin/bash


TEMP=$1


if [ $TEMP -gt 5 ]; then
        if [ $TEMP -lt 15 ]; then
                echo "The weather is cold."
        elif [ $TEMP -lt 25 ]; then
                echo "The weather is nice."
        else
                echo "The weather is hot."
        fi
else
        echo "It's freezing outside ..."
fi
```

Using Case statement in bash

```
case "variable" in
        "pattern1" )
```

```
                Command … ;;
        "pattern2" )
                Command … ;;
        "pattern2" )
                Command … ;;
esac
```

Pay attention!

- The patterns are always followed by a blank space and **)**.
- Commands are always followed by double semicolon **;;**. White space is not mandatory before it.
- Case statements end with **esac** (reverse of case).

```
#!/bin/bash

CHAR=$1

case $CHAR in
[a-z])
echo "Small Alphabet." ;;
[A-Z])
echo "Big Alphabet." ;;
[0-9])
echo "Number." ;;
*)
echo "Special Character."
esac
```

Test conditions in bash

Condition	Equivalent
$a -lt $b	$a < $b
$a -gt $b	$a > $b
$a -le $b	$a <= $b
$a -ge $b	$a >= $b
$a -eq $b	$a is equal to $b
$a -ne $b	$a is not equal to $b
-e $FILE	$FILE exists
-d $FILE	$FILE exists and is a directory.
-f $FILE	$FILE exists and is a regular file.
$STRING1 = $STRING2	$STRING1 is equal to $STRING2
$STRING1 != $STRING2	$STRING1 is not equal to $STRING2

Let's create one final script named filetype.sh that detects whether a file is a regular file, directory:

```
#!/bin/bash


if [ $# -ne 1 ]; then
        echo "Error: Invalid number of arguments"
        exit 1
fi


file=$1


if [ -f $file ]; then
```

```
        echo "$file is a regular file."
elif [ -d $file ]; then
        echo "$file is a directory."
else
        echo "$file does not exist"
fi
```

Exercise 7:

Write a shell script that accepts a positive integer as argument and checks if the number is odd or even and prints 'Number X is odd' or 'Number X is even' accordingly.

If there is no argument or more than two arguments or a negative number provided, display an error message: "You have to provide one non negative number".

Use modulo operator % to check if reminder is 0 or not. $# gives the number

Output: For number 182, the output should be "Number 182 is even" and for input 23, the output should be "Number 23 is odd"

Loops in Bash

For Loops in Bash

```
for i in {1..10}; do
        echo "Hello Friend"
done
```

While Loops in Bash

```
#!/bin/bash


num=1
while [ $num -le 10 ]; do
```

```
        echo $(($num * 3))
        num=$(($num+1))
done
```

Until Loops in Bash

The key difference between until loop and while loop is in the test condition. A while loop will keep running as long as the test condition is true; on the flip side, an until loop will keep running as long as test condition is false!

```
#!/bin/bash


num=1
until [ $num -gt 10 ]; do
        echo $(($num * 3))
        num=$(($num+1))
done
```

Try the following:

```
#!/bin/bash


prime=(2 3 5 7 11 13 17 19 23 29)
for i in "${prime[@]}"; do
        echo $i
done
```

```
for ((i=1;i<=10;i++)); do
```

```
        echo $i
        if [ $i -eq 3 ]; then
                break
        fi
done
```

```
#!/bin/bash
for ((i=0;i<=10;i++)); do
        if [ $(($i % 2)) -ne 1 ]; then
                continue
        fi
        echo $i
done
```

Exercise 8:

Output all the files and directory that exists under the /var directory.

Use /var/* as the range of the for loop to increment through.

Using Functions in Bash

Creating functions in bash

```
function_name () {
commands
}
```

```
#!/bin/bash
```

```
hello () {
echo "Hello World"
}

hello
hello
hello
```

Passing arguments to bash function

```
#!/bin/bash

iseven () {
if [ $(($1 % 2)) -eq 0 ]; then
echo "$1 is even."
else
echo "$1 is odd."
fi
}

iseven 3
iseven 4
iseven 20
iseven 111
```

```
#!/bin/bash


fun () {
echo "$1 is the first argument to fun()"
echo "$2 is the second argument to fun()"
}


echo "$1 is the first argument to the script."
echo "$2 is the second argument to the script."


fun Yes 7
```

Local and global variables in bash functions

```
#!/bin/bash


v1='A'
v2='B'


myfun() {
local v1='C'
v2='D'
echo "Inside myfun(): v1: $v1, v2: $v2"
}


echo "Before calling myfun(): v1: $v1, v2: $v2"
myfun
echo "After calling myfun(): v1: $v1, v2: $v2"
```

Recursive functions

```
#!/bin/bash


factorial () {
if [ $1 -le 1 ]; then
echo 1
else
last=$(factorial $(( $1 -1)))
echo $(( $1 * last ))
fi
}


echo -n "4! is: "
factorial 4
echo -n "5! is: "
factorial 5
echo -n "6! is: "
factorial 6
```

Exercise 9:

Write a shell script that accepts a number and check if it is prime or not. Using function is must here.

Prime numbers are only divisible by 1 and the number itself. 23 is prime, 21 is not prime. 1 and 2 are also considered prime.